Ders 2 Genişletilmiş Notu: Bilgisayarın Yapısı, Komut Çevrimi, Verinin 0-1 ile Kodlanması ve Algoritmaya Giriş

Ders 2 Genişletilmiş Notu: Bilgisayarın Yapısı, Komut Çevrimi, Verinin 0-1 ile Kodlanması ve Algoritmaya Giriş

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Bilgisayar Nedir?
- 2. Bir Bilgisayarın Temel Bileşenleri
- 3. Program ve Proses
- 4. Bus'ların Görevi
- 5. Komut Çevrimi
- 6. Bilgi Neden 0 ve 1 ile Kodlanıyor?
- 7. Yarı İletkenler, Diyot ve Transistör
- 8. Transistör Yoğunluğunun Sınırı: R-C Etkisi
- 9. Her CPU'nun Kendi Komut Seti ve Mimarisi Vardır
- 10. Bellekler ve Hızları
- 11. Darboğaz Örneği: Laboratuvar Bilgisayarları
- 12. Von Neumann ve Harvard Mimarileri
- 13. İşlemcide 0 ve 1'lerin Kullanımı
- 14. Makine Dili, Assembly ve Programlama Dilleri
- 15. Veri Türleri ve Kodlanmaları
- 16. Görüntü, Ses, Video ve Diğer Veriler
- 17. Bit, Byte ve Birimler
- 18. Boyut Hesapları
- 19. Örnekleme ve Nicemleme
- 20. Sinüzoidaller, Tını ve Örnekleme Teoremi
- 21. Gerçek Dünya Problemleri ve Algoritma
- 22. Algoritmanın Özellikleri
- 23. Kek Tarifi Örneği
- 24. Üç Temel Yapı

1. Bilgisayar Nedir?

Bu ders, birinci dönemdeki Bilgisayar Bilimlerine Giriş dersinde görülen konuların, algoritmayı da içine alan bir tekrarıdır. Hoca makinenin en temelinden başlayıp yavaş yavaş gerçek dünya problemlerine doğru ilerledi. Bazı konuların sayısal analiz dersinde de konuşulduğunu, ama burada bu dersin ruhuna uygun olarak sıfırdan ele almanın daha doğru olduğunu söyledi. Ayrıca bu derste kullanılacak akış diyagramı (flowchart) notasyonunun diğer kaynaklardan biraz farklı olabileceğini ve problemlerin bu notasyona uygun çözülmesi gerektiğini hatırlattı.

Hocanın ilk sorusu şuydu: Bir bilgisayar mühendisi olarak bilgisayardan ne anlamalıyız? Bilgisayar aslında bir hesaplama aracıdır. Normal bir insandan farkı, bizim de yapabildiğimiz hesaplama işlemini daha hızlı ve hatasız yapması, bir de çok büyük verileri saklayabilmesidir.

Burada “hatasız” sözü önemlidir. Bilgisayar, **tarif ettiğimiz** hesaplama işlemi neyse onu hatasız yapar. Hoca bunu şöyle gösterdi: İki sayıyı toplamak istiyorsunuz ve bir kod yazıyorsunuz. Ama kodda yanlışlıkla şunu yazmışsınız:

```
x = a * b;
```

Bilgisayar bu çarpma işlemini hatasız yapar. Sizin beklediğiniz sonuç farklı olabilir, ama bu bilgisayarın hatası değil, sizin hatanızdır. Demek ki bilgisayarın özellikleri şunlardır: hatasız işlem yapabilmesi, çok büyük verileri saklayabilmesi ve bizim çok uzun sürede yapabileceğimiz işleri çok kısa sürede tamamlayabilmesi.

2. Bir Bilgisayarın Temel Bileşenleri

Peki bilgisayar veri saklama ve işlem yapma işini kendi içinde nasıl hallediyor? Hoca öğrencilere her bilgisayarda olması gereken bileşenleri sordu.

Soru: Bir bilgisayarda olması gereken temel bileşenler nelerdir? Öğrencilerden üç cevap geldi: işlemci, RAM, sabit disk ve bunları birleştiren anakart; giriş birimleri, çıkış birimleri ve işlem birimi; bir de güç kaynağı.

Cevap: Hoca ilk cevabın kişisel bilgisayara yakın olduğunu, oysa temel bir bilgisayar deyince bir mikroçipi de düşünebileceğimizi söyledi. Giriş-çıkış birimleri cevabını “biraz yakın” buldu. Güç kaynağının varlığından ise zaten şüphe yoktur.

Bir bilgisayar sisteminde olması gereken üç temel birim şunlardır:

- **İşlem birimi (CPU):** Programları koşturur.
- **Ana bellek (main memory, birinci bellek):** CPU’nun doğrudan veri alabildiği bellektir. Program ve veri (data) saklar. Bunu daha çok RAM olarak görebiliriz, ama illa doğrudan RAM olmak zorunda değildir.
- **Bus yapısı:** CPU ile bellek arasındaki haberleşmeyi ve veri transferini sağlar. Bu yapı tek değildir, üç tanedir.

Hoca aslında dört bileşen olduğunu, ama dördüncüsünün temel bir bilgisayar sisteminde zorunlu olmadığını söyledi. Dördüncü yapı, bus üzerine bağlanan çeşitli birimlerdir. Bunlar giriş birimi de çıkış birimi de olabilir. Bunlara **çevresel birimler (peripheral devices)** denir ve genellikle giriş-çıkış (I/O, input-output) işleri için kullanılırlar.

CPU’da temel olarak yapılan iş programların koşturulması, yani komutların icra edilmesidir (execution). “Çalıştırır” veya “icra eder” de diyebiliriz. “Koşturmak” denmesinin sebebi, programın satır satır, peş peşe çalışan bir şey olmasıdır. Hatta bazı geliştirme ortamlarında programı çalıştırmak için koşan adam simgesi gösterilir. Execution’ın doğrudan çevirisi icradır, yani bir şeyin gereğini yerine getirmektir.

3. Program ve Proses

CPU’da koşacak program nereden gelir? Ana bellekten. Koşturulacak olan programın ve verilerinin saklandığı yer burasıdır. Hocanın sorusu, koşturulmakta olan programın özel adıydı. Bu ad **prosestir** (process, “program in execution”). Hoca Ctrl+Alt+Del ile görev yöneticisini açıp bir sürü programın adını ve her birinin ID’sini gösterdi.

Program ile proses birbirinden farklı şeylerdir:

- **Program**, bizim anladığımız şeydir. İçinde kod yazdık, for döngüsü, while döngüsü var. Bunu biz anlarız, bilgisayar anlamaz. Birkaç adımda bilgisayarın anlayacağı dile dönüştürülür ve çalışabilir hâle getirilir.
- **Proses**, çalışan programdır. Çalışabilir dosyanın üzerine çift tıkladığınızda ya da komut satırından adını yazdığınızda program proses hâline gelir. Bu sırada programın çalıştırılabilir hâlinin bir örneği belleğe yüklenir. Bellekte bir yer kaplar, kendine ait verileri ve program kodları vardır.

Sonra her bir komut, birinci satırdan itibaren tek tek CPU’ya getirilir, CPU’da bazı işlemlerden geçirilir ve program böylece koşturulur. Bu getirme aşamasında kullanılan yapı bus’tır.

Soru: Yazılan kodu CPU’nun önce derlemesi (compile) mi gerekiyor?

Cevap: Hayır, CPU derlemez. Derleyici (compiler) derler ve kodu çalıştırılabilir (executable) hâle getirir. Bu çalıştırılabilir hâli biz çalıştırdığımızda program proses hâline gelir. CPU ise bu prosesi tek tek icra eder, yürütür. Yani CPU'da derleyicinin bir işi yoktur.

4. Bus'ların Görevi

Bus yapıları şunlardır: **adres bus**, **data bus** ve **control bus**.

> **Hocanın vurgusu:** Bir bilgisayar sisteminde temelde kaç tür bus vardır? Bu, ileride mülakatlarda ve bazı sınavlarda çok karşılaşacağınız bir sorudur. Cevap üçtür: adres, data ve control.

Görevlerini görmek için CPU'nun bir komut işleyeceğini düşünelim. CPU işleyeceği komutun bellekteki adresini bilir. Bunun için program counter (PC) ve instruction register gibi özel register'lar vardır. Program counter, "git 1 numaralı adresteki veriyi getir" der. Yani getirilecek komutun nerede olduğunun bilgisini tutar.

Bellekten okuma şöyle olur:

- Control bus, bellekten okuma yapılacağını söyler. Okunacak şeyin ne olduğu burada önemli değildir. Örneğimizde işlemcinin direktifi gereği bir komut okunacaktır.
- Adres bus'ın gösterdiği yerden okuma yapılır.
- Okunan veri, data bus üzerinden işlemcinin içine getirilir.

Bu yolun gerekme sebebi şudur: İşlemcinin içinde çok büyük bir hafıza yoktur, belli bir kapasitesi vardır. Bu yüzden yanına bir bellek konur ve program ile veri, adres bus'ın gösterdiği yerden data bus yoluyla işlemciye getirilir. Hoca burada ana belleğe "yardımcı bellek" dedi, yani işlemciye yardımcı olan bellek anlamında kullandı.

Belleğe yazma ise tersine çalışır. Bir komut çalıştı, diyelim ki `x = a * b` yazdık, sonra `x = x + 1` gibi bir komut geldi. X'in güncel değerinin belleğe kaydedilmesi gerekebilir. Bu durumda:

- Control bus bir yazma sinyali gönderir.
- Yazılacak veri data bus üzerinden gider.
- Yazılacak adres adres bus üzerinden gider ve bellekte yazma işlemi gerçekleşir.

Bunun detayları bilgisayar donanımı, mikroişlemci ve belki assembly derslerinde görülecektir.

5. Komut Çevrimi

Peki işlemci bir komutu nasıl işler? Komut, bus'lar üzerinden gelecek, ama işlem aşamalı bir şekilde olur. Hoca bu soruyu öğrencilere sordu. Bir öğrenci, program belleğe yüklendikten sonra işlemcinin counter'ı sıfırlayarak baştan başlayıp komutları tek tek aldığını söyledi. Hoca bunu doğru buldu, ama asıl sorusunun "komut çalışacak" kısmının içi olduğunu belirtti: Bir komut nasıl icra edilir, burada hangi adımlar vardır? Başka bir öğrenci kontrol birimine ve aritmetik birime yönlendirme fikrinden söz etti, bu "yakın ama tam değil" bulundu. Bir diğeri fetch, store, execution gibi adımları andı. Hoca burada doğru kelimeyi yakaladığını, yalnızca sırasının farklı olduğunu söyledi.

Komut çevriminin (instruction cycle) adımları şunlardır:

1. **Fetch instruction:** Komut gidip getirilir. Adres bus üzerinden yer bulunur, data bus üzerinden komut getirilir.
2. **Decode instruction:** Getirilen 0 ve 1'lerden oluşan komut çözümlenir. İşlemci komutun ne olduğunu anlar.
3. **Execute instruction:** Komutun gereği yerine getirilir, örneğin toplama yapılır.
4. **Write back:** Sonucun yazılması gerekiyorsa belleğe yazılır.
5. Bir sonraki komut için tekrar fetch adımına dönülür.

Bu döngü, program beklenmedik şekilde durana (halt) ya da programın sonuna gelinene (termination) kadar devam eder.

> **Hocanın vurgusu:** Fetch "getirmek" değil, "gidip getirmek"tir. Çünkü önce gideceğimiz yeri bilmemiz gerekir. Bu yeri işlemcinin içindeki bir register, genellikle program counter söyler.

Decode adımını biraz açalım. Getirdiğimiz komut 0 ve 1'lerden oluşur, çünkü C programı işlemcinin makine diline göre derlenmiştir. Bu dizinin toplama mı, çıkarma mı olduğunun çözümlenmesi gerekir. Komutun içinde iki tür bilgi vardır:

- **Opcode (operation code):** Yapılacak işleme ait bilgi. Örneğin ADD, SUB.
- **Operandlar:** Operasyonun yardımcı elemanları. Yani ne ile ne toplanacak, sonuç nereye yazılacak.

  Ders sonrası not: Hoca bu yapıyı "operation code şu, artı operandlar" diye anlatırken makine koduna da opcode dedi; doğrusu büyük olasılıkla şudur: Komutun tamamı makine kodudur, opcode ise yalnız işlemi belirten kısımdır.

Adımların detayları başka derslerde gelecektir. Getirme işini bus'lar yapar. Execution, özellikle toplamanın toplama devreleriyle nasıl yapıldığı Lojik Devreler dersinde anlatılır. Write back ise getirmenin bir anlamda tersi gibi çalışır.

6. Bilgi Neden 0 ve 1 ile Kodlanıyor?

CPU yarı iletkenlerden oluşan bir cihazdır ve her şey elektriksel işaretlerle olur. Bilgisayar sistemleri elektrikle çalışır. Kullandığımız bilgisayarlarda çoğunlukla 0 ile 12 volt veya 0 ile 5 volt aralığında bir voltaj vardır. Ama aradaki değerleri kullanmak yerine yalnızca 0 ve 1 kullanırız. 0, 0 volta karşılık gelir. 1 ise sisteme göre 5 volta ya da 12 volta karşılık gelir. Peki neden?

Soru: Neden çok seviyeli bir kodlama yerine 0 ve 1 kullanılıyor?

Cevap: Öğrenciler üç gerekçe söyledi. Seviyeler arası geçiş zaman alır, örneğin 8'den 5'e düşmek ile 10'a çıkmak arasında zaman farkı olur. Manyetik ve elektrik alanlar gibi dış etkiler, seviyeler sıksa bir seviyeyi başka bir seviyeye kaydırabilir. Hassasiyeti artırıp hatayı azaltmak gerekir. Hoca bunları yerinde buldu ve bir örnekle açtı.

6.1 Haberleşme Protokolü ve Orta Bahçe Benzetmesi

Aramızda şöyle bir protokol yaptığımızı düşünelim: 0 volt, 1 volt, 2 volt, hatta arada buçuklu voltlar da ekleyerek 12 volta kadar seviyelerimiz olsun. Kablonun bir ucunda ben varım, öbür ucunda elinde voltmetre olan bir arkadaş. Şu seviyeyi gönderirsem A, bunu gönderirsem B, şunu gönderirsem C göndermiş olurum. Ara seviyeleri de eklersek daha çok veri kodlayabiliriz. Yani iki kişi arasında gizli bir haberleşme sistemi kurduk. Şimdi ben B gönderdiğimde karşı tarafın B okuma olasılığı nedir? Bunun için karşıda 1 volta yakın bir değer ölçülmesi gerekir.

Hoca bunu üniversitenin orta bahçesiyle anlattı. Bir kişi bahçenin kapı tarafında, hoca da yemekhane tarafında duruyor. "Bu parmağımı kaldırırsam 1 anla, bunu kaldırırsam 2 anla" desek, o mesafeden hangi parmağın kalktığı anlaşılamaz. Gözün algılaması o mesafe için yetersizdir, yani kayıp olur.

Elektrikte de durum aynıdır. İletkenin direnci, ısı ve benzeri etkiler yüzünden gönderilen 1 volt karşıda 1 volt okunamaz, belki 0,5 volt okunur. Eğer 0,5 voltu da başka bir bilgi olarak kodladıysam karşı taraf yanlış mesajı görür. Oysa bahçede "elim havadaysa A, aşağıdaysa B algıla" ya da "üzerimde kırmızı varsa bir şey, yoksa başka bir şey algıla" deseydim, mesaj karşıya doğru ulaşırdı. Bir şeyin **varlığı veya yokluğu** ile bilgi temsil etmek çok daha doğru bir kodlama sağlar. Dijital kodlama sistemi, verinin analog kodlanmasında ortaya çıkan bu bozucu etkiler yüzünden ortaya atılmıştır. Analog kodlama da vardır, ama bilgisayarlarda henüz tam olarak kullanamıyoruz, çünkü bir sürü problemi vardır.

6.2 Eşikler ve Kararsızlık Kuşağı

Dijital sistem elektriğin varlığına veya yokluğuna bakar. Hoca bir voltaj ekseni çizdi: Bir uçta 0 volt, diğer uçta 12 volt var. Belli bir noktaya kadar ölçülen voltaj değerleri 0 bilgisine, belli bir noktadan sonra ölçülen değerler 1 bilgisine karşılık gelir. Örneğin 3 volt ve 7 volt seçelim:

- 3 volta kadar bir değer gelirse bu 0 olarak algılanır.
- 7 volt ve üzerinde bir değer gelirse bu 1 olarak algılanır.

- Aradaki değerler, örneğin 4 veya 5 volt, kararsızlık durumudur. Bu bölgeye **kararsızlık kuşağı** denir.

Gönderilen 12 volt karşı tarafa doğrudan gitmez, bir kayıpla gider. İletkenin direnci ne kadar fazlaysa kayıp da o kadar fazladır, çünkü güç kaybı olur ve enerji ısıya gider. Demek ki burada malzeme bilgisi de önemlidir. Kararsızlık kuşağı ne kadar kısaysa o kadar iyidir. Bu konular Lojik Devreler dersinde daha ayrıntılı görülecektir.

6.3 Varlık ve Yokluk da Tam Çözüm Değil

Soru: Kuantum bilgisayarlarda bir şeyin hem 1 hem 0 olma ihtimalinden yararlanılıyor. Bu da voltajla ilgili bir şey mi?

Cevap: Hayır, orası farklıdır. Hoca bunların gelecekte önümüzde beklediğimiz şeyler olduğunu, bu sorunun da söyleyeceği bir şeye iyi bir giriş olduğunu belirtti.

Elektriğin varlığını veya yokluğunu saklamak da tam bir çözüm değildir, hız açısından bizi doyuma ulaştırmaz. Çünkü elektriğin 0 volttan 12 volta çıkması belli bir zaman alır. Üstelik tek bir bit kodlamıyoruz. Milyarlarca, trilyonlarca bit kodlanıyor ve her birinin bu seviyeye çıkarılması zaman alıyor. 12 volttan 5 volta inilmesinin bir sebebi de budur: 5 volta çıkmak daha kısa sürer.

7. Yarı İletkenler, Diyot ve Transistör

Elektriğin varlığını ve yokluğunu en iyi temsil eden malzemeler yarı iletkenlerdir. En çok bilinenleri silisyum ve germanyumdur. Bunun detayları fizikçi gözüyle yarı iletken fiziği dersinde anlatılır, hoca burada özet olarak aktardı.

Temel kavram **bariyer voltajıdır**. Bariyer voltajı uygulanana kadar malzeme yalıtkandır, bu voltaj geçildiğinde iletken hâle gelir. Teorik olarak tahta bile böyledir: Tahtanın bariyer voltajı çok yüksek olduğu için elektronları harekete geçirmek zordur ve tahta elektriği hemen hemen hiç iletmez, ama çok yüksek bir gerilim verilirse iletimin sağlanacağı söylenir. Hoca, yarı iletkenlerde bu değerleri silisyum ve germanyum için yaklaşık 0,7 ve 0,3 volt olarak hatırladığını söyledi. Bu malzemelerden üretilen elektronik devre elemanlarıyla veri kodlanır.

Yarı iletkenlerden önce de kondansatörlerle veri saklamak mümkündü. Kondansatörü doldurur, yani şarj edersiniz, zamanla belli bir miktarda deşarj olur. İçinden ölçtüğünüz voltaj belli bir seviyenin üzerindeyse burada 1 vardır, altındaysa (örneğin 3 voltun altına inmişse) 0 vardır dersiniz. Fakat kapasitesinin fazla olması gibi olumsuzluklar yüzünden kondansatörler tek başına tam anlamıyla işe yaramaz. Yarı iletken malzemeler daha faydalıdır.

Diyot, bunlardan üretilen en çok kullanılan elemanlardan biridir. Uçlarına bir gerilim uygularsınız. Gerilim, kullanılan malzemenin bariyer voltajından fazlaysa iletken hâle gelir, azsa yalıtkan kalır. O zaman akım geçiyorsa 1, geçmiyorsa 0 olduğunu düşünebiliriz.

Transistör, diyotun yetersiz kaldığı durumlarda devreye girer. Hoca buna bir benzetme sordu ve cevabı kendisi verdi: **musluk**. Musluğu sonuna kadar sıkarsanız su akmaz, yani akımı sıfırlarsınız, direnci neredeyse sonsuz yaparsınız. Transistör de bir gerilim bilgisini ayarlayarak akımı sıfırlayan veya çok artıran, yarı iletkenlerden yapılmış ve bilginin 0 ve 1 olarak kodlanmasını sağlayan bir devre elemanıdır. Bunu Ohm yasasıyla da görebiliriz:

$$V = I \times R \quad \Rightarrow \quad I = \frac{V}{R}$$

Direnç sonsuz olursa $V/\infty = 0$ olur, yani akım sıfırdır. Direnç sıfıra indirilirse teorik olarak akım sonsuz olur, yani musluğu sonuna kadar açtığınızda akabildiği kadar akar. Aradaki değerler akım kontrolü için kullanılır.

Transistörün emitter, base ve kolektör uçları ve bunlardan geçen akımlar arasındaki ilişkiler Elektronik Devreler dersinde anlatılır. Biz işin analog tarafında değiliz. Bizi ilgilendiren, akımın geçip geçmediği, yani bilginin 0 mı 1 mi olduğudur. Hoca görselden de gösterdi: Transistör üç bacaklı, elle tutulabilen bir devre elemanıdır ve farklı şekilleri vardır, alan etkili olanları da

vardır. Yapısı aslında iki diyotun bir şekilde bir araya getirilmesiyle oluşur. İşlemcinin içindekiler ise bunlar değil, mikro transistörlerdir.

Transistörlerin icadı ile işlemcinin içinde bir sürü mikro transistör birbirine bağlanarak mantıksal devreleri ve hafıza elemanlarını oluşturur: AND ve OR kapıları, flip-flop'lar, register'lar. Toplama, çıkarma, çarpma ve bölme gibi işlemler bu elemanlarla, 0 ve 1'ler üzerinden yapılır. Detayları Lojik Devreler dersindedir.

8. Transistör Yoğunluğunun Sınırı: R-C Etkisi

Transistör ideal bir eleman mıdır? Değildir. İşlemci sınırlı bir alana sahiptir ve bu küçük alana çok sayıda mikro transistör sığdırılır. Hoca güncel i7 ve i9 işlemciler için 40-50 milyon civarında bir sayı okuduğunu söyledi.

> **Ders sonrası not:** Hoca güncel işlemcilerde 40-50 milyon transistör olduğunu söyledi; doğrusu büyük olasılıkla milyarlar mertebesindedir.

Bu transistörlerin hepsi aktif ya da pasif, yani 1 ya da 0 hâlindedir. 1 hâlindeyken akım geçirirler. Malzemenin içinde belli bir direnç vardır, akım geçince bir gerilim ve ısı oluşur. Her transistörün ısısı birikir. İşlemcilerin soğutulma sebebi budur. Üzerine termal macun gibi malzemeler sürülür, soğutucu ve fan konur. Sunucu ortamlarında bir sürü soğutucu kullanılır. Soğutucu devreden çıktığında ise işletim sistemi donanıma zarar gelmesin diye bilgisayarı otomatik olarak kapatır. Hoca bu durumu bir darboğaz (bottleneck) olarak adlandırdı.

Transistörler arasındaki yatay ve düşey aralıklarla oynayarak sığdırılacak sayıyı değiştirebiliriz. Düz mantıkla, ne kadar sık yerleştirirsek o kadar çok transistör sığdırırız. Ama burada iki etki ortaya çıkar:

- **Çok sıkıştırırsak R etkisi:** Bir odaya 80 kişiyi sığdırmaya çalıştığınızı düşünün, sıcaklık artacaktır. Buna rezistif etki, yani direnç ve ısı etkisi denir.
- **Çok açarsak C etkisi:** Hem daha az transistör sığar, hem de iki iletken arasına bir yalıtkan girmiş olur. İki iletken levha arasına hava gibi bir yalıtkan koyduğunuzda kondansatör oluşur. Buna C etkisi denir.

Çok sıkıştırmak R etkisi, çok açmak C etkisi getirdiği için buna **R-C etkisi** denir. Bunun optimum hesapları vardır. Bir orta yol bulmak gerekir. Bu duruma **trade-off (ödünleşim)** denir: Bir şeyden ödün verirken başka bir şey kazanırsınız. Hocaya göre R-C etkisi, bilgisayar bilimlerinde ve elektronikte karşılaşılan trade-off'lardan biridir. Bu yüzden günümüz bilgisayarları çok hızlı olsa da gelmesi gereken yere gelemiyor.

> **Ders sonrası not:** Hoca sık yerleşimde R, seyrek yerleşimde C etkisi ortaya çıktığını söyledi; doğrusu büyük olasılıkla farklıdır. İletkenler inceldikçe direnç artar, birbirlerine yaklaştıkça aralarındaki kapasite de artar. Yani sıkıştırma hem R'yi hem C'yi büyütür ve sınırı bunların oluşturduğu gecikme ile ısı birlikte koyar.

Soru: Bu anlatılanlar x86 için mi geçerli? ARM ile x86 arasındaki fark nedir?

Cevap: Hoca belirli bir markadan değil, bu mantıkla çalışan genel işlemcilerdeki mimariden söz ettiğini söyledi. ARM işlemcilerdeki geliştirme mantığı ile Intel veya AMD işlemcilerdeki geliştirme mantığı farklıdır. Bu fark, RISC ve CISC diye anılan komut seti mimarilerinden ve Harvard ile von Neumann gibi geliştirme mimarilerinden kaynaklanır. Bu bir mikroişlemciler konusudur.

> **Ek bilgi (derste anlatılmadı):** RISC büyük olasılıkla az sayıda ve basit komut kullanan, CISC ise daha çok sayıda ve daha karmaşık komut kullanan komut seti yaklaşımını anlatır. ARM işlemciler büyük olasılıkla RISC, x86 işlemciler ise CISC geleneğindendir. Bu ayrımın ayrıntıları Mikroişlemciler dersinin konusudur.

Elektriği bu şekilde tutmak bir yere kadar iyidir, ama bir yerden sonra maliyetli ve problemlidir. Hoca küresel ısınma açısından da konuya değindi: Bir sürü sunucu çalışıyor ve bunları soğutmak gerekiyor. Soğutma klima etkisiyle havayı kurutur. Hoca bunu akciğerin nemini kaybetmesine benzetti.

> **Ders sonrası not:** Hoca sunucu soğutmasının zararını havanın kurutulması üzerinden anlattı; asıl çevresel maliyet büyük olasılıkla soğutma ve çalıştırma için harca-

nan enerjidir. Akciğer benzetmesini bilimsel bir açıklama olarak değil, bir benzetme olarak alın.

Bu yüzden farklı çalışmalar vardır. Kuantum bilgisayarlar bunlardan biridir. Bir diğeri optik bilgisayarlardır: Elektrik yerine ışığın varlığını ve yokluğunu bilgi temsil etmek için kullanmak. Örneğin aynalar ve prizmalardan oluşan bir yapıda bir ışık hüzmesi döner durur. Işık varsa bir bilgiye, yok olmuşsa 0'a karşılık gelir. Bunlar daha çok malzeme, elektronik ve elektrik mühendislerinin çalışma alanıdır. Hoca, mekatronikten bir hocanın hard disk malzemesinin hız açısından önemini ve SSD'lerden daha hızlı olacak bir sistem üzerine Japonya'da yapılan bir çalışmayı anlattığını da aktardı.

Buraya kadarın özeti şudur: Verinin varlığının veya yokluğunun işlemci biriminde saklanması, bütün verileri simgelememizi ve komutları oluşturmamızı sağlar.

9. Her CPU'nun Kendi Komut Seti ve Mimarisi Vardır

> **Hocanın vurgusu:** "Each CPU (processing unit) has its own instruction set and architecture." Bu cümle çok önemlidir, not edilmeli ve unutulmamalıdır. Her CPU kendine özgü bir komut setine ve mimariye sahiptir.

Herhangi bir firmanın (ARM, Intel, AMD) bir modelini ele alıp data sheet'ine baktığınızda bir sürü komut görürsünüz. Hoca birkaçını araştırmanızı önerdi, çünkü bunlara rahatça erişilebilir. Başka bir modelde bu komutlar da mimari de farklıdır. Örneğin üçüncü nesil, dördüncü nesil denen farklar kısmen buradan gelir: Birinde 3,8 voltla, birinde 3,7 voltla anahtarlama yapılıyor gibi. Bu küçük farklar önemlidir, çünkü milyonlarca, milyarlarca transistörün anahtarlanması zaman açısından çok önemlidir.

10. Bellekler ve Hızları

0 ve 1'leri yalnız CPU'da kullanmıyoruz. Buraya kadar anlatılanların hepsi RAM için de, hatta hard disk için de geçerlidir. Bilgi her yerde bir şeyin varlığı veya yokluğuyla kodlanır, yalnızca bunu sağlayan fiziksel yol değişir.

- **RAM:** Kullandığımız dinamik RAM'ler transistör ve kondansatörlerle oluşturulur. Hoca, statik RAM'lerde ve cache belleklerde daha çok transistör kullanıldığını bildiğini söyledi. Bilgi yine elektriğin varlığına veya yokluğuna göre kodlanır.
- **Hard disk:** Eski, silindirli yapıdaki disklerde bilgi manyetik olarak yazılır. Okuma, manyeto-rezistif direnç denen bir kafayla yapılır. Kafa oradan bir voltaj ve akım bilgisi okuyabiliyorsa orada 1, okuyamıyorsa 0 vardır. Bu daha yavaş bir süreçtir, çünkü okunan bilginin analogdan dijitale dönüştürülmesi gerekir.
- **CD ve DVD:** Burada optik olarak yakma gerçekleşir. Işıkla veri yakılır, bunlara pit ve land denir. Bir CD'nin yazılmış kısımları farklı renkte görünür. Boş CD ile dolu CD bu yüzden rahatça ayırt edilir.

  > **Ders sonrası not:** Hoca hard diskte de bir "yakma" gerçekleştiğini söyledi; doğrusu büyük olasılıkla yüzeydeki küçük bölgelerin manyetik yönünün değiştirilmesidir. Yakma CD ve DVD için geçerlidir.

Hard disk ayrıca sürekli dönen bir yapıdır. Günümüz hard diskleri 7200 veya 5400 rpm (rotation per minute) ile çalışır. Hoca 7200 rpm üzerinden şu hesabı yaptı:

- Verilen: Dakikada 7200 dönüş.
- İşlem: $7200/60 = 120$, yani saniyede 120 dönüş.

> **Ders sonrası not:** Hoca buradan $1/120 \approx 1/100$ alıp "0,01 Hz" dedi; doğrusu saniyede 120 dönüş 120 Hz'tir. $1/120$ saniye ise bir dönüşün süresidir, yaklaşık 8,3 ms eder.

Sonra RAM hızına geçildi. Bir öğrenci 3200 MHz söyledi. Hoca bunu MHz mertebesi olarak, işlemcileri de 3,2 GHz gibi GHz mertebesi olarak yerleştirdi. Bu hız sıralaması şöyledir: En hızlısı işlemci, ondan yavaşı RAM, ondan da çok çok yavaşı hard disk.

> **Ders sonrası not:** Hoca 3200 MHz RAM ile 3,2 GHz işlemci arasında 10^3 kat fark olduğunu söyledi; doğrusu 3200 MHz = 3,2 GHz olduğu için bu iki sayı aynıdır. Bel-

leğin işlemciden yavaş kalması büyük olasılıkla erişim gecikmesi ve bus üzerindeki aktarım süresinden kaynaklanır.

11. Darboğaz Örneği: Laboratuvar Bilgisayarları

Hoca bölümdeki bir laboratuvardan gerçek bir örnek verdi. Laboratuvarda Core 2 Duo işlemcili bilgisayarlar vardı. Bu makinelerden istenen iş azdı: Yalnız C programlama derslerinde kullanılacaklar, en fazla Java, belki PostgreSQL çalıştıracaklardı. Makinelerin arızası yoktu, ama yavaştılar ve öğrenciler şikâyet ediyordu.

Soru: Bu bilgisayarları hızlandırmak için ne yapılır?

Cevap: Öğrencilerden gelen cevap sabit diskleri değiştirmek, SSD takmaktı. Hoca bunu doğru buldu. RAM'i artırmak burada işe yaramaz. Çok işlem gerektiren bir iş, örneğin oyun çalıştıracaksak RAM'in 4 GB yerine 8 GB olması fayda sağlar. Ama C veya Java çalıştıracak bir bilgisayarda RAM'in 2, 4 ya da 16 GB olması pek fark etmez, çünkü RAM dolmuyor. 2 GB RAM'de bir C programı en fazla 1 MB gibi bir yer tutar, yani RAM'in yaklaşık 2000'de biri kadar. Rahat rahat sığar.

Darboğaz hard diskti. Makinelere SSD takılınca, bir buçuk dakikada açılan bilgisayarlar 10-15 saniyede açılmaya başladı. İşlemci ve bellek hızı, bu kadar basit kullanım için zaten fazlasıyla yeterliydi.

Hoca bunu şöyle benzetti: Çok hızlı bir otomobil alıp köy yolunda, stabilize ya da toprak bir yolda gidiyorsunuz. 180-190 yapamazsınız, belki 40-50 ile gidersiniz. Yapılması gereken, üzerinde gidilen yolu, yani sistemin en yavaş birimini iyileştirmektir. İyileştirilebilecek başka şeyler de vardır, örneğin bus'ların da kendine göre hızları vardır. Ama bu laboratuvarda hard diski düzeltmek sorunu büyük ölçüde çözmüştü.

12. Von Neumann ve Harvard Mimarileri

Bir önceki konuda çizilen yapıda işlemci, ana bellek, bus ve çevresel birimler vardı. Bu yapıda komutlar ve veriler ana bellekte bütünleşik olarak saklanıyorsa buna **von Neumann mimarisi** denir. Eğer CPU'nun program belleği (program memory) ile veri belleği (data memory) birbirinden ayrılırsa, yani ikisi de bellek olduğu hâlde CPU'nun bunlara erişimi farklı kanallardan olursa buna **Harvard mimarisi** denir.

Harvard yapısında işlemci bir yandan programı, bir yandan veriyi alır. Von Neumann mimarisine göre bus'lar daha etkin kullanılır, yani kod alınırken veriler de hazırlanabilir. Bu ayrım çok eski bilgisayar mimarilerinde yapılan bir şeydir. Günümüzde "şu şundandır, bu bundandır" diyebileceğiniz kesin bir ayrım yoktur. Firmalar (hocanın saydıkları: Microchip, ARM teknolojileri, Intel, AMD, Panasonic, Motorola) iki mimarinin üstünlüklerini alarak daha iyi tasarımlar yapmaya çalışır.

13. İşlemcide 0 ve 1'lerin Kullanımı

Peki işlemcide 0 ve 1'ler nasıl kullanılır? İki temel kullanım vardır:

- **Instruction set encoding:** 0 ve 1'leri belli bir mantıkla bir araya getirip "buna ADD diyeceğim" derim. Komut setimde kaç komut olmasını istiyorsam her birine ait bir 0-1 dizisi olur. Dikkat edin, buradaki decoding değil encoding'dir. Komut setinin encode edildiği yer, işlemcideki kontrol ünitesidir.
- **Instruction execution:** Daha çok önemli olan budur. Bütün komutların görevi (performs all of instructions) neyse yerine getirilir. Toplama yapılacaksa toplama yapılır. Matematiksel ve mantıksal işlemler **ALU (aritmetik lojik ünite)** içinde yapılır.

ALU'da toplama **lojik seviyede** yapılır (addition in logic level). Yani girdi olarak verilen 0 ve 1'lerden normal bildiğimiz toplamayı yapacak devreler kurulur. Bunlara toplama devresi denir. Çıkarma da toplama cinsinden yapılabildiği için toplama-çıkarma devreleri de denir. Bu devreler yine yarı iletkenlerden oluşan lojik kapılarla (logic gates) kurulur. Detayları Lojik Devreler, yani digital design dersinde görülecektir.

Komut çevrimi de bunlara göre yapılır: Komut getirilir, encode edilmiş komut çözülür, execute edilir ve sonuç gerekirse belleğe yazılır. Ana bellekte de veriler aşağı yukarı aynı mantıkla saklanır.

14. Makine Dili, Assembly ve Programlama Dilleri

Buraya kadar anlatılanlara bakılırsa bir problem çözmek için CPU'nun komut setindeki bütün komutları ezberlemek, RAM'in ve bus'ların nasıl çalıştığını en ince ayrıntısına kadar bilmek gerekiyor gibi görünüyor. Hoca bunun makul olup olmadığını sordu.

Soru: Bir problemi çözmek için işlemcinin bütün komut setini ezberlemek makul mü?

Cevap: Öğrenciler makul olmadığını söyledi. Birden fazla platformda çalışmak isteyebiliriz ve her platform için ayrı geliştirmek zaman alır. Bütün komutları ezberlemek de bir insan için çok zordur. Hoca bunları doğruladı. Komut seti işlemciden işlemciye değişir. Mikrodenetleyici veya gömülü sistem dersi alırsanız görürsünüz: PIC16F84 eğitimde çok kullanılan bir entegredir ve bundan PIC16F877'ye geçişte bile büyük farklar vardır. Çok sayıda işlemci varken bunları ezberleyemeyiz. Ezberlesek bile açıp 0 ve 1'ler yazarak veya assembly ile iş yapamayız. Çok alt seviye programlama, gömülü yazılım geliştirme gibi durumlarda yapılabilir, ama genel olarak makul değildir.

Buradan iki tür dile geliriz:

- **Makine dili:** Üzerinde çalıştığımız donanıma bağımlıdır (hardware dependent). İşlemci değişirse bu dil de değişir.
- **Assembly dili:** Makine dilinin doğrudan karşılığıdır (eşdeğeri, hocanın sözüyle ekivalenti). O da donanıma bağımlıdır.

Bunlarla program yazmak hem taşınabilirlik hem de rahat kodlama açısından zordur. Bir web sayfası yapacaksak kimse bu işi assembly'de yapmaz. Etkileşimin fazla olduğu, kullanıcıya hitap eden veya donanıma doğrudan erişim gerektirmeyen işler için başka bir katman gerekir.

- **Programlama dilleri:** Donanımdan bağımsızdır (hardware independent) ve günlük konuşma dilimize çok daha yakındır.

14.1 Kaynak Koddan Çalışan Programa

Süreç şöyle işler:

1. Problemi bir programlama diliyle kodlarım ve bir kaynak kod (source code) oluşur: `.c`, `.py`, `.java`, `.cs` gibi.
2. Derleyici veya yorumlayıcı denen yardımcı programlar bu kaynak kodu nihai olarak makine koduna çevirir. Arada başka adımlar da vardır.
3. Elde edilen makine kodu, çalıştırılabilir (executable) koddur. Makinenin doğrudan çalıştırabileceği bir koddur. Kaynak kod 10 byte ise bu dosya 10 kilobyte gibi çok daha büyük olabilir, çünkü işin içine ek kütüphaneler ve başka değerler girer.
4. Çalıştırılabilir programı çift tıklayarak veya komut satırından açtığımda proses hâline gelir ve işlemcide yürütülür.

Demek ki donanımla konuşabilmek ve problemlerimizi ona tarif edebilmek için bir programlama dili kullanmamız gerekir.

Soru: Python da doğrudan makine koduna mı derleniyor?

Cevap: Hayır. Onlar yorumlayıcılı (interpreter) dillerdir, adım adım dönüştürülürler. Kaynak kod önce **bytecode** denen bir ara koda çevrilir. Çünkü işletim sistemi de işin içine ek bir faktör olarak girer. Bu ara kod sonra Java'da Java Virtual Machine, Python ve C#'ta da kendilerine ait mekanizmalarla, üzerinde çalışılan işletim sisteminin özelliklerinden geçirilerek makine koduna dönüştürülür. Yani doğrudan "pat" diye bir dönüşüm yoktur.

Derleyici veya yorumlayıcı kaynak kodu ayrıştırırken sözdizimi (syntax) analizi, anlam (semantic) analizi gibi analizler yapar. Bunun ayrıntıları Programlama Dillerinin Kavramları, Hesaplama Kuramı ve Derleyici Tasarımı gibi derslerde anlatılır.

14.2 Komut Setinde Olmayan İşlem

Programlama dilinde yaptırmak istediğimizi donanıma tarif ederiz. Donanım bunu kendi komut repertuarındaki komutlarla gerçekleştirir. Bir çarpma işlemi işlemcimde yoksa onu diğer işlemler cinsinden tarif edebilmem gerekir. Toplama olmasaydı onu da tarif etmem, gerekirse bit seviyesinde yazmam gerekirdi. Neyse ki toplama, çıkarma, çarpma, bölme, atama ve karşılaştırma gibi temel işlemler hemen hemen her komut setinde bulunur.

Hocanın örneği karşılaştırmaydı. 8086 işlemcilerde CMP (compare) adında bir komut vardır. İki register'ı karşılaştırır ve bize bir değer döndürür. PIC'lerde böyle bir komut yoktu, karşılaştırma için çıkarma veya başka bir mekanizma kullanmak gerekiyordu. Alt seviyede, assembly ile bunları yazmak daha zordur. Programlama dilleri bu yüzden büyük kolaylık sağlar.

Programlama dillerinin neden bir anda değil yavaş yavaş yaygınlaştığına da değinildi. Bilgisayarlar önce askerî, sonra sağlık ve stratejik alanlarda kullanıldı, sonra kişisel olarak yaygınlaştı. Etkileşim ihtiyacı insanlarda bir anda ortaya çıkmadı. Zamanla gereksinimler değiştikçe makineler de farklı hizmetler sunmaya başladı.

> **Ders sonrası not:** Hoca 30 yıl önce internet olmadığını söyledi; doğrusu büyük olasılıkla o dönemde internet vardı, ama bugünkü gibi yaygın değildi.

15. Veri Türleri ve Kodlanmaları

Veriyi 0 ve 1'lerle temsil edeceğimizi konuştuk, ama veri dediğimiz şey nedir? Günlük hayatta veriler çoğunlukla analog dünyadadır. Hocanın saydıkları şunlardır:

- Metin (text) ve karakterler.
- Sayılar: tam sayılar (integer), kayan noktalı sayılar (floating point) ve karmaşık sayılar.
- Boolean ya da binary veriler.
- Görüntü, ses (sound, voice) ve video.
- Herhangi bir fiziksel büyüklük, yani her türlü veri (any type of data).

> **Ders sonrası not:** Hoca metin verilerini de analog veri başlığı altında saydı; doğrusu büyük olasılıkla metnin karakterlerden oluşan ayrık bir veri olduğudur. Ses veya sıcaklık gibi sürekli bir büyüklük değildir.

Bunların hepsinin 0 ve 1'lerle, belli bir sistematikle kodlanması gerekir.

15.1 Karakterler

Karakter deyince büyük A'dan Z'ye, küçük a'dan z'ye harfler, soru işareti gibi noktalama işaretleri (punctuation) ve klavyedeki farklı simgeler anlaşılır. Bunların bir sözlük mantığıyla 0-1 dizileri hâline getirilmesi gerekir ve bu rastgele yapılmaz, bir mantıkla yapılır. ASCII ve Unicode bu işi belli bir sistematikle yapan yapılardır. Hocanın verdiği örnek şudur:

Karakter	Ondalık	İkili
A	65	`01000001`
a	97	`01100001`
B	66	`01000010`
b	98	`01100010`

Hoca 65'i $2^6 + 1 = 64 + 1$ olarak açtı. Tahtada başta sondaki 1 yazılmamıştı, bir öğrenci uyarınca düzeltildi. Büyük harf ile küçük harf arasında yalnız bir bit fark vardır: $97 - 65 = 32$, yani +32'lik bir fark. $98 - 66 = 32$ de aynı sonucu verir (ara adım eklendi).

Hoca, Shift veya Caps Lock'a basınca arka planda bu bitin değiştiğini söyledi. Klavyeden girilen her tuş 0 ve 1'e dönüştürülür ve bu dönüşüm çok gecikme ve maliyet oluşturmayacak bir mantıkla tasarlanır.

> **Ders sonrası not:** Hoca Shift tuşuna basınca bu bitin değiştiğini söyledi; bu büyük olasılıkla sezgisel bir anlatımdır. Klavye büyük olasılıkla tuşun kodunu gönderir, karaktere çevirme işini yazılım yapar. Büyük ve küçük harf arasındaki 32 farkı ise ASCII tablosunda gerçekten vardır.

Karakterleri bitiştirerek (concatenation) metinleri, yani karakter katarlarını (string) oluştururuz. Aradaki boşluk da bir karakterdir.

15.2 Tam Sayılar

Tam sayılarda (integer) iki bilgi saklanır: işaret bilgisi ve büyüklük bilgisi. Diyelim ki 8 bitlik bir bellek elemanımız var. Hangi hücreyi işaret için kullanacağız? Mantıken ortadaki bir hücreyi de seçebiliriz, ama bu sağlıklı değildir. Çünkü aradaki diğer bitleri ayrıştırmak, maskelemek gerekir. Bu yüzden ya en baştaki ya da en sondaki bit seçilir. Bu kodlamayı kullananlar **MSB (most significant bit, en anlamlı bit)** kısmını işaret biti (sign bit) olarak almıştır:

- İşaret biti 1 ise sayı negatiftir.
- İşaret biti 0 ise sayı pozitiftir.
- Geri kalan bitler, 8 bitlik örnekte 7 bit, büyüklük bilgisini taşır.

C'de `int` 4 byte, yani 32 bittir. Bir bit işarete gittiği için aralık şöyledir:

$$-2^{31} \leq \text{int} \leq 2^{31} - 1$$

Bu aralık yetmiyorsa 8 byte ya da 16 byte kullanılabilir.

> **Ders sonrası not:** Hoca işaret biti ile büyüklük bitlerini anlattı, sonra aralığı -2^{31} ile $2^{31} - 1$ olarak verdi. Doğrusu büyük olasılıkla şudur: İşaret-büyüklük gösteriminde aralık $-(2^{31} - 1)$ ile $2^{31} - 1$ olur ve iki ayrı sıfır çıkar. Tahtadaki aralık ise bilgisayarların kullandığı ikiye tümleyen gösterimine aittir. Orada da en anlamlı bit 1 ise sayı negatiftir.

15.3 Kayan Noktalı Sayılar

Kayan noktalı sayılara örnek ±3,25 gibi değerlerdir. Yine bir işaret biti vardır. Ayrıca mantissa ve exponent alanları bulunur. Hoca, bir alanın virgülden önceki kısmı, diğerinin virgülden sonraki kısmı tuttuğunu söyledi ve detayları belki sonraki derslerde örneklerle göstereceğini belirtti.

> **Ders sonrası not:** Hoca mantissa ve exponent alanlarının virgülden önceki ve sonraki kısımları tuttuğunu söyledi; doğrusu büyük olasılıkla şudur: IEEE 754'te sayı $(-1)^{\text{işaret}} \times 1{,}\text{mantissa} \times 2^{\text{exponent}-\text{bias}}$ biçiminde saklanır. Exponent ikinin kuvvetini, mantissa anlamlı basamakları tutar. Single precision'da 1 işaret, 8 exponent ve 23 mantissa biti vardır.

Bu kodlamanın mantığı sistemden sisteme değişebilir, bu yüzden bir standart olması gerekir. Hoca burada notasyon ile standardı ayırdı. Matematikte aynı işlemi yapan birkaç farklı notasyon görebilirsiniz. Ama standart, herkes tarafından aynı şekilde uygulanmak zorundadır. Bu iş için **IEEE 754** kayan nokta standardı çıkmıştır. İki versiyonu vardır:

- **Single precision:** 32 bit.
- **Double precision:** 64 bit. Daha hassastır.

Hassasiyet önemlidir, çünkü virgüllü sayılarda ne kadar hassas olursak hatayı o kadar azaltırız. Sayısal analizde de görülecektir. Hoca örnek olarak şunu verdi: Mayın tarıyorsanız %90 başarı sıkıntılı olabilir. Müzik üzerinde işlem yapıyorsanız %80 veya %85 başarı çok önemli olmayabilir.

> **Ders sonrası not:** Hoca %90 başarıda "o %1 problem" dedi; doğrusu kalan oran $100 - 90 = \%10$'dur.

15.4 Karmaşık Sayılar ve Boolean

Karmaşık sayılar iki alandan oluşur. İkisini de birer kayan noktalı sayı olarak saklayıp ikincisinin önünde bir i varsayarsam $a + bi$ biçiminde bir karmaşık sayı kodlayabilirim. Boolean ise doğrudan 0 ve 1'in kendisidir, false ve true olarak saklanır.

Bunlar **ilkel (primitive) veri tipleridir**. Doğrudan bir sayı veya bir metin içeren hemen hemen her büyüklüğü bunlarla ifade edebiliriz. Ama yalnız bunlardan yola çıkarak her problem doğrudan çözülemez. Örneğin bir görüntüyü saklamak için bunların üzerine başka bir yapı kurmak gerekir.

16. Görüntü, Ses, Video ve Diğer Veriler

16.1 Görüntü

Görüntü, satır ve sütunlardan ve bunların kesişimindeki **piksel** denen elemanlardan oluşan bir veridir. Satır ve sütun sayısına çözünürlük denir. Satır ve sütun ne kadar çoksa çözünürlük de o kadar fazladır. Kamera, lens yardımıyla analog veriyi örnekleyerek dijitale dönüştürür. Örnekleme, analog bir büyüklüğü belli bir kayıpla simgelemek ve saklayabilmek demektir.

Bir pikselde olabilecek en büyük sayıyı pozitif tam sayılarla belirleriz. 0'dan 255'e kadar kodlama yaparsam 256 farklı değer olur ve bu 8 bitle halledilir. Kabul şudur: 0 simsiyah, 255 bembeyaz. Aradaki değerler siyahtan beyaza bir geçiştir. Buna **gri seviye** denir.

Bellek, yani RAM, tek boyutlu bir yapıdır. İsterseniz görüntü, isterseniz video saklayın, sonuçta her şey tek boyutlu olarak saklanır. Görüntüyü bir matris gibi düşünürsek (0,0) pikseli belleğin başında saklanır, (0,1) onun ardından gelir. M satır ve N sütun varsa sonuncusu $(M-1, N-1)$ pikselidir. Toplam $M \times N$ eleman çözünürlüğü verir. Çözünürlük arttıkça gerçek dünyadan daha fazla örnekleme yapılmış demektir, ama kapasite de artar.

İkinci parametre, her pikseldeki bit sayısıdır. Bu sayı 8, 16, 32 ya da daha fazla olabilir. Bit sayısı arttıkça renk geçişleri çeşitlenir. Böylece görüntü boyutu şu olur:

$$\text{boyut} = M \times N \times (\text{piksel başına bit})$$

Hocanın örneği: 10 satır, 10 sütun, 8 bit gri seviye bir görüntü (böyle bir görüntünün gerçekçi olmadığını da belirtti).

- İşlem: $10 \times 10 \times 8 = 800$ bit.
- Byte'a çevirme: $800/8 = 100$ byte.

16.2 Ses

Ses de ister müzik ister insan konuşması olsun, tek boyutlu bir dizi olarak saklanır. Görüntü iki boyutludur, ses tek boyutludur. Sesinizi kaydedip dalga formuna bakarsanız bir dalga görürsünüz. Bu dalgadan belli bir **örnekleme frekansına** göre örnekler alınır. Görüntüdeki çözünürlüğün sesteki karşılığı budur.

Bunun için standartlar belirlenmiştir. CD kalitesi için örnekleme frekansı $f_s = 44100$ Hz'tir. Yani saniyede 44.100 örnek, yani dizi elemanı alınır. Hoca bunu insan kulağının duyabildiği aralığa bağladı. İnsan kulağının duyabildiği en yüksek frekans 20 kHz'tir, yani 20.000 Hz. Örnekleme frekansının bunun en az iki katı olması gerekir: $2 \times 20000 = 40000 \leq 44100$ (ara adım eklendi). YouTube'da duyma testleri vardır, düşük frekanstan yüksek frekansa doğru ilerler, 20 kHz'e varmadan duyma kesilir.

> **Ders sonrası not:** Hoca insan kulağının duyma aralığını "2 Hz ile 20 kHz" olarak verdi, "2 kilohertz'de duymaya başlarsınız" da dedi; doğrusu büyük olasılıkla yaklaşık 20 Hz ile 20 kHz'tir.

Hoca ayrıca örnekleme frekansını 22050 Hz'e, yani yarıya çevirmeyi videoları 2x hızda izlemeye benzetti: Örnek atladıkça ses hızlı çalıyor, daha da azaltırsanız kulak algılayamıyor.

> **Ders sonrası not:** Hoca 2x izlemenin örnekleme frekansını düşürmek olduğunu söyledi; bu büyük olasılıkla yalnız kısmen doğrudur. Örnek atlayıp aynı hızda çalmak sesi hızlandırır, ama bir dosyayı düşük örnekleme frekansıyla kaydedip o frekansta çalmak onu hızlandırmaz, yalnız yüksek frekans bileşenlerini kaybettirir.

Dört dakikalık CD kalitesinde bir müziğin boyutu: Hoca bu hesabı şöyle kurdu.

- Süre: $4 \times 60 = 240$ saniye.

- Eleman sayısı: Her saniyede 44.100 örnek olduğu için $240 \times 44100 = 10.584.000$ eleman (sonuç ara adım eklendi). Görüntüdeki $M \times N$'in karşılığı budur, bellekte bu kadar hücre ayrılır.
- Her hücrenin çözünürlüğü: Hoca 10 bit örneğini verdi. Sesin dalga formunda negatif değerler de olduğu için aralık -2^9 ile $2^9 - 1$ arasıdır, yani -512 ile 511 arası (sayısal değerler ara adım eklendi).
- Toplam: 10 bitle $10.584.000 \times 10 = 105.840.000$ bit eder, bu da $13.230.000$ byte'tır (ara adım eklendi).

Örnek başına bit sayısı artırılırsa sesin kalitesi artar. Bilgisayarlarda bunu değiştiren uygulamalar vardır, ama bir yerden sonra işe yaramaz. Çünkü ses kartının da bir çözünürlüğü vardır ve genellikle 16 bit kullanılır. Ses kartının işleyebileceği yere kadar artırabilirsiniz.

16.3 Video ve Diğer Veriler

Video, birden fazla görüntünün ve gerekirse bunlara eklenen bir sesin birleştirilmesidir. Yani matrisle sesi birleştirirsiniz. Hoca saniyede en az 24 görüntü (frame) olduğunu hatırladığını söyledi. Bu 24 frame'in değişimine uygun bir ses dosyası bir araya getirilir.

Bunun dışında her türlü veri benzer şekilde saklanır. Bir aracın hızının zamana göre değişimini kaydedip nerede hızlandığını, nerede yavaşladığını inceleyebilirsiniz. Bu, sese benzeyen tek boyutlu bir veridir. Benzer şekilde bir büyüklüğün dönemlere ya da derslere göre değişimi de bir veridir. Bunların hepsi bir veya birden fazla boyuta göre değişen verilerdir.

17. Bit, Byte ve Birimler

Bir görüntünün, sesin ya da metnin boyutu sık sorulan bir sorudur. Bunun için önce birimleri bilmek gerekir.

Bir bit ya 0 ya 1'dir, yani iki durum saklar: var veya yok. Örneğin yoklama yapılacak olsa her öğrencinin varlığını bir bitle saklayabilirim. Ama numaraları saklamak istersem bir bit yetmez, daha fazla bit gerekir. 4 bitin bir araya gelmesine **nibble**, 8 bitin bir araya gelmesine **byte** denir. Bir byte da çok yeterli olmadığı için birimler büyür:

- $2^{10} = 1024$ byte = 1 kilobyte (KB). Yani buradaki "kilo" 2^{10} demektir.
- 2^{10} KB = 1 megabyte (MB) = $2^{10} \times 2^{10} = 2^{20}$ byte.
- 2^{10} MB = 1 gigabyte (GB) = 2^{30} byte.
- Terabyte ve devamı (peta, zeta gibi) aynı mantıkla gider.

  Ek bilgi (derste anlatılmadı): Derste kilo 2^{10} olarak kullanıldı. Standartlarda ise kilo büyük olasılıkla 1000 anlamına gelir ve 2^{10} için KiB gibi ayrı semboller tanımlanmıştır. Disk üreticileri ve ağ hızları büyük olasılıkla 1000 tabanını kullanır. Bu yüzden aynı boyut farklı yerlerde biraz farklı görünebilir.

Soru: Megabit için küçük b, megabyte için büyük B mi yazıyoruz?

Cevap: Evet. Küçük "b" bit, büyük "B" byte demektir. Kilobit, gigabit de aynı şekildedir. İnternet hızlarında saniyede aktarılan gigabit miktarı söylenir (gigabit per second).

18. Boyut Hesapları

18.1 16 GB RAM Örneği

Bir öğrencinin bilgisayarında 16 GB RAM varmış. İşlemci bu belleği adresleyecek. Az önce dedik ki komutu bir adresten getireceğiz, öyleyse her byte'a bir adres vermemiz gerekir: 0. byte, 1. byte, 2. byte ve böyle son byte'a kadar. 16 GB'ı kaç bitle kodlayabiliriz?

- $16 = 2^4$ ve 1 GB = 2^{30} byte'tır.
- Toplam: 16 GB $= 2^4 \times 2^{30} = 2^{34}$ byte.
- Her byte'ı göstermek için **34 bit** gerekir. 0. byte'ın adresi 34 tane 0, son byte'ın, yani $(2^{34} - 1)$. byte'ın adresi 34 tane 1'dir. Aradaki adresleri bir ekleyerek bulursunuz.

Hoca konuşurken bir yerde "34 tane byte" dedi, ama kastedilen 34 bittir. RAM üzerindeki adresleme hemen hemen böyle işler.

Bu bellekte kaç bit var? Bellekte 2^{34} hücre var ve her hücre 8 bit ise:

$$2^{34} \times 8 = 2^{34} \times 2^3 = 2^{37} \text{ bit}$$

Bitler tek tek adreslenmez, genellikle byte byte adresleme yapılır. Bu, belleğin kaç bitlik hücrelerle organize edildiğine bağlıdır. Bellek 8 bitlik hücrelerle organize edilmişse 2^{34} hücre vardır ve her birinin içindeki 8 bite ayrıca adres verilmez. Belleğin içindeki elektronik devre sayısına bakarsanız bu kadar, belki daha fazla yarı iletken malzeme kullanılır.

18.2 Stereo ve Aynı Notadaki Farklı Enstrümanlar

Bu hesaplardan sonra ses konusunda iki soru geldi.

Soru: Ses tek boyutlu bir dizide saklanıyorsa, farklı müzik aletlerini nasıl ayırt edebiliyoruz?

Cevap: Hoca önce soruyu sinemadaki yönlü ses gibi anladı. Sağdan gelip soldan geçen bir araba sesi, birden fazla kanal demektir. Birden fazla kanal, birden fazla dizi demektir. Stereo kayıtta iki dizi, daha kaliteli kayıtlarda 3-4 dizi olur. Video ile eş zamanlılık da burada önemlidir: 24 frame ile 44.100 Hz'lik sesin uyumlu olması gerekir. Olmazsa konuşan kişinin ağız hareketleri sesle uyuşmaz ve bu insanı rahatsız eder.

Soru: Stereoyu değil, gitarda ve piyanoda aynı frekansta çalınan notayı nasıl ayırt ettiğimizi soruyorum.

Cevap: Buna **tını** denir. Hoca asıl cevabı örnekleme konusundan sonra vereceğini söyledi. Bu cevap Bölüm 20'dedir.

18.3 Metin Boyutu Örneği

Hocanın verdiği problem şudur: Bir metin 100 karakterden oluşuyor ve bu metinde 50 farklı karakter (distinct characters) kullanılmış. Metin en az ne kadar yer kaplar? Dikkat edin, ASCII kullanılacağı söylenmedi.

- 100 karakter varsa 100 elemanlı bir dizi olacak. C üzerinden konuşursak 0. karakterden 99. karaktere kadar.
- Her karakterin kaç bit tutacağını bulmalıyız. 50 farklı karakteri kaç bitle temsil edebilirim? Koşul: $2^n \geq 50$, burada n bit sayısıdır.
- n için 1, 2 ve 5 olmaz. $2^5 = 32$ yetmez, $2^6 = 64$ yeter (sayısal değerler ara adım eklendi). Yani en az **6 bit** gerekir. Daha fazla da olabilir, ama burada istenen asgari bellektir.
- Hangi karaktere hangi 0-1 dizisinin geleceği burada önemli değildir. Önemli olan karakter başına bit sayısıdır.
- Toplam: $100 \times 6 = 600$ bit.

Hoca bunu birime çevirmek için yaklaştırma yaptı: $100 \approx 2^7 = 128$ ve $6 \approx 2^2 = 4$, yani $2^7 \times 2^2 = 2^9 = 512$. Bu 600'den küçük kaldığı için 2^{10}'a kadar gidilebileceğini söyledi.

> **Ders sonrası not:** Hoca 600 bit için "yarım kilobayt ile 1 kilobayt arası" dedi; doğrusu 2^{10} bit 1 kilobit'tir, kilobayt değildir. 600 bit, $600/8 = 75$ byte eder.

18.4 Görüntü Boyutu Örneği

Problem: 512×512 bir görüntüde her piksel 400 farklı renkten birini alabiliyor. Hoca gerçekte 400 renk olmayacağını, örneğin 65536 gibi değerler görüleceğini söyledi, ama hesap için önemli değil.

- Eleman sayısı: 512×512 piksel. Yani $2^9 \times 2^9 = 2^{18}$.
- Piksel başına bit: $2^8 = 256$ yetmez, $2^9 = 512$ yeter. Yani 9 bit.
- Toplam: $2^9 \times 2^9 \times 9 = 2^{18} \times 9$ bit.

Ders sonrası not: Hoca 2^{18}'i 2^{20}'ye yaklaştırıp "yaklaşık 9 megabit, yaklaşık 1 megabayt" dedi; doğrusu $2^{18} = 2^{20}/4$ olduğu için bu fazla kaba bir yaklaşımdır. Tam değer $2^{18} \times 9 = 2.359.296$ bit, $2.359.296/8 = 294.912$ byte, $294.912/1024 = 288$ KB'tır. Bu da 2,25 megabit eder.

19. Örnekleme ve Nicemleme

Ara sonrasında hoca, makinenin en temelinden başlayıp gerçek dünyaya doğru ilerlediğimizi hatırlattı. Temel veri tiplerini nasıl saklayacağımızı gördük. Şimdi analogdan dijitale geçişin arka planına bakalım. Herhangi bir analog veriye genellikle **sinyal** deriz: Zamana göre değişen bir $x(t)$ fonksiyonu veya işareti. Bu bir ses olabilir, havanın sıcaklığı olabilir, bir dövizin gün içindeki seyri olabilir, dersteki katılımcı sayısının saniye saniye değişimi olabilir.

Soru: Bu veriyi bilgisayarda olduğu gibi doğrudan saklayabilir miyiz?

Cevap: Bir öğrenci saklayamayacağımızı, bitlere indirgememiz ve kesikli olarak noktalar hâlinde saklamamız gerektiğini söyledi. Hoca bir adım daha sordu: Her t değeri için $x(t)$ değerini biliyorum, bunların hepsini kayan noktalı sayı olarak saklayabilir miyim? Bir başka öğrenci, sonsuz bilgi çıkacağını ve belli bir aralık bulmak gerektiğini söyledi. Hoca bu cevabı doğru buldu.

Analog sinyal, sonsuz detay içerir, zaman içinde sonsuz aralık vardır. Oysa bilgisayarın belleği, işlemcisi ve bütün kapasite elemanları sonludur. Bu yüzden sonsuz durumdan sonlu duruma geçmemiz gerekir. Geçiş iki eksende olur. Genel adı **ayrıklaştırmadır (discretization)**.

19.1 Örnekleme (Sampling)

Zaman ekseninde yapılan ayrıklaştırmaya **örnekleme (sampling)** denir. Belli zaman aralıklarında değer alınır ve aradaki bilgiler kayıpla bırakılır. Elde ettiğimiz şey artık $x(t)$ değil, $x[n]$ biçiminde bir dizidir. t yerine n gelmiştir ve n bir dizinin indisidir. C'de `int a[100];` deyip n'inci elemana bakmak gibi. n_{max} değerine göre bellek tahsisi (allocation) yapılır, yani n_{max} tane verimiz olur. İndis 0'dan başlar, birer birer gider.

19.2 Nicemleme (Quantization)

Peki dizinin her elemanı hangi aralıkta değer alacak? Genlik (amplitude) ekseninde yapılan ayrıklaştırmaya **nicemleme (quantization)** denir. Sinyalin 0 ile en büyük değeri (max) arasındaki aralık seviyelere bölünür. Her seviyeye bir 0-1 dizisi verilir: En alt seviye tamamen 0'lardan, en üst seviye tamamen 1'lerden oluşur. Böylece hem her hücrenin kaç bit tutacağını hem de hangi değerin hangi 0-1 dizisine karşılık geleceğini bulmuş oluruz. Koşul şudur:

$$\text{aralık} \leq 2^m, \quad m = \text{bit sayısı}$$

Hoca burada n ile karışmasın diye bit sayısına m dedi. Her eleman m bit içereceği için toplam boyut şöyledir:

$$\text{boyut} = n_{max} \times m \text{ bit}$$

Özetle:

- **Zamanda ayrıklaştırma:** sampling.
- **Genlikte ayrıklaştırma:** quantization.

19.3 Ayrıklaştırmanın Bedeli

Her iki eksendeki ayrıklaştırma da orijinal veride kayba neden olur. Bu kayıp, ses veya görüntü söz konusuysa insan kulağının veya gözünün ayırt edemeyeceği, rahatsız olmayacağı düzeyde tutulmalıdır. Hoca seçim anketlerini de örnek verdi: Anketlerin istatistiksel tekniklere uygun, belli bir örnek sayısıyla yapılması gerekir. Örnekleme görüntüde, müzikte, konuşmada, her yerde vardır.

20. Sinüzoidaller, Tını ve Örnekleme Teoremi

Şimdi öğrencinin sorusuna dönelim: Analog sinyaller nasıl birbirine karışmıyor, bir kayıtta enstrümanlar nasıl ayrı kalıyor?

20.1 Sinüzoidallerin Toplamı ve DTMF

Her sinyal teorik olarak sinüzoidal sinyallerin toplamı cinsinden yazılabilir. Hoca telefonunun tuşlarına basarak her tuşun farklı bir ton verdiğini dinletti. Bu farklılık, her sesin farklı temel sinüzoidaller içermesinden kaynaklanır. Telefon tuşları iki sinüzoidalle kodlanmıştır. Buna **DTMF** denir. Örneğin:

- 1 tuşu: 697 Hz'lik bir sinüs dalgası ile 1209 Hz'lik bir sinüs dalgasının toplamıdır.

Birinin frekansı diğerinden yüksektir. İkisini bir araya getirmek, ikisini toplamak gibidir ve oluşan toplam 1 tuşunun sesini verir. Bunun bir de genlik bilgisi vardır.

20.2 Tını

Hoca sonra aynı notanın farklı enstrümanlarla çalınışını gösterdi ve dinletti. Aynı nota farklı enstrümanlarda farklı dalga formları verir. Buna **tını (timbre)** denir. Örneğin C6 notası (bizde Do notası diye geçen ailede, burada C1, C2 gibi kodlanır) yaklaşık 1047 Hz'lik bir sinüs dalgasıdır. Frekans düzleminde karşılığı da yaklaşık 1047 Hz'te tek bir bileşendir. Aynı nota obuayla çalındığında yine 1047 Hz vardır, ama yanında ek bileşenler de görünür. Klarnette ise farklı bir yapı görünür. Bunların birbirinden farklı olan bu karakteristik özelliği tınıdır.

Soru: Yani enstrümanların bir notasında sabit tek bir frekans olmadığı ve bunlar bir araya gelince kendine özgü bir şey oluştuğu için mi ayırt edebiliyoruz?

Cevap: Enstrümanları ayırt etmek ayrı bir mevzudur, recognition konusuna, makine öğrenmesine girer. Her enstrümanın bir temel frekansı ve ekstra frekans bileşenleri vardır. C6'da 1047 Hz sabittir, ama bunu insan sesiyle ya da başka bir enstrümanla verdiğinizde ekstra frekanslar işin içine girer. Örneğin hocanın gösterdiği bir kayıtta çalan kişi biraz fazla üflediği için asıl frekansı bastıran ek bir bileşen oluşmuştu. Flütte de elin yanlış tutulup hava kaçması farklı bir ses çıkarır. Hoca bunlara "gürültü diyebiliriz" dedi.

> **Ders sonrası not:** Hoca enstrümanların ek frekans bileşenlerine "gürültü" dedi; doğrusu büyük olasılıkla bunların çoğu temel frekansın harmonikleridir ve enstrümanın tınısını oluşturur. Hava kaçması gibi istenmeyen sesler ise gerçekten gürültüye daha yakındır.

20.3 Örnekleme Teoremi

Karışmama sorusunun cevabı **örnekleme teoremidir**: Örnekleme frekansı, işaretin içindeki en yüksek frekanslı bileşenin **en az iki katı** olmalıdır. Hoca bunu CD örneğine bağladı. 44100 Hz'in, insan kulağının üst sınırının iki katından biraz fazla olarak seçilmesi bundandır. Bir ses kaydedici, müzikteki bütün enstrümanların ulaştığı en yüksek frekansı hesaba katmalı, onun en az iki katı frekansla örnek almalıdır. Örnekleme iki kattan düşük yapılırsa sinyal geri çatılırken (reconstruction) enstrümanlar birbirine karışır.

> **Ders sonrası not:** Hoca 02:27:58 civarında "2 ile 20 hertz" ve "20 hertz'in iki katı" dedi; kastettiği, Bölüm 16.2'deki gibi 20 kilohertz'tir. Ayrıca hoca enstrümanların karışmamasını örnekleme teoremine bağladı. Doğrusu büyük olasılıkla şudur: Teorem, frekans bileşenlerinin geri çatmada birbirine karışmasını önler. Bir kayıttaki enstrümanların kulakla ayırt edilmesi ise tını ile ilgilidir.
>
> **Ek bilgi (derste anlatılmadı):** İki katın sebebi büyük olasılıkla şudur: Bir sinüs dalgasının her periyodundan en az iki örnek alınmazsa o dalga geri çatıldığında daha düşük frekanslı başka bir dalgayla karıştırılabilir. Bu olaya büyük olasılıkla örtüşme (aliasing) denir. Örnekleme teoremi kaynaklarda büyük olasılıkla Nyquist-Shannon adıyla da geçer.

Hoca tek enstrümanlı ve çok enstrümanlı kayıt örnekleri de dinletti. Aynı notayı insan sesinin de üretebileceğini söyledi: Do notasını insan sesi de bir enstrüman da aynı hertz değerinde

üretebilir. Son olarak ses üretimine değindi. "A" dediğinizde, sesin gırtlaktan çıkıp ağızdan üretilmesine kadar geçen sürede ses yolunda bir tepe noktası oluşur, sonra azalır. İki kez "a" derseniz iki tepe, üç kez derseniz üç tepe görülür. Bu çizim sesin kendisi değil, konuşma üretim modelinin bir gösterimidir. Bu konular sayısal işaret işleme ve sinyaller dersleriyle ilgilidir.

21. Gerçek Dünya Problemleri ve Algoritma

Gerçek dünyada veriler üzerinde işlem yapılır (data processing). Bilgisayara bir girdi veririz ve bir çıktı olarak yeni bir veri üretiriz. Bazen veri bize verilir, bazen onu biz oluştururuz.

Gerçek dünya problemlerinin (real life problems) ortak yapısı şudur: Size giriş verileri verilir veya bunları siz bulursunuz. Bu girişlerden bir çıkış elde edilmesi istenir. Sizin de giriş ile çıkış arasındaki fonksiyonu yazmanız gerekir.

```
giriş verileri  --->  f  --->  çıkış
```

Hoca **havuz problemini** örnek verdi. Bir havuzda bazı musluklar dolduruyor, bazıları boşaltıyor ve bunların çalışma hızlarını biliyoruz. Havuzun ne kadar sürede dolacağını veya boşalacağını hesaplamak istiyoruz. Girişlerimiz dört tanedir: Dolduran musluklar $+t_1$ ve $+t_2$, boşaltan musluk $-t_3$ ve havuzun hacmi. Çıkış ise havuzun dolma süresi t'dir. Aradaki f fonksiyonuna sistem modeli, sistem fonksiyonu, davranış fonksiyonu veya transfer fonksiyonu denir. Bu dersteki asıl işimiz bu f'nin tasarlanmasıdır.

Bu fonksiyonun üretilmesiyle ilgili daha da önemli bir kavram vardır: **algoritma**. Algoritma, verilen giriş verilerinden çıkış verilerinin üretilmesini sağlayan fonksiyonu bulma sürecidir.

Bu her zaman matematiksel bir ifade değildir, yani illa x^2+y^2 gibi bir fonksiyon olmak zorunda değildir. Örneğin N elemanlı bir sayı dizisi girersiniz ve en büyüğünü ararsınız. Bunun bir girişi (N eleman) ve bir çıkışı (var, yok ya da bulunan değer) vardır. Bu geliştirme süreci de bir algoritmadır.

Algoritmanın gerçek bir bilgisayar ortamında hayata geçirilmesine **gerçekleştirme (implementation)** denir ve bunun karşılığı programdır. Algoritma görece soyuttur, programlama somuttur. Kafanızda bir problemin çözümüne ilişkin bir taslak oluşturursunuz. Bu taslağı kurallarına uygun şekilde bir programlama dilinde yazarsınız. Bu da programlama sürecidir.

22. Algoritmanın Özellikleri

Hoca algoritmanın üç temel özelliğini verdi. Bir algoritma analizi veya tasarımı kitabına baktığınızda bu kelimeleri göreceksiniz:

- **Unambiguous** (açık, belirsizlik içermeyen). Kitaplarda "clear" da denebilir.
- **Finite** (sonlu).
- **Sequence and set of instructions** (sıralı komutlar kümesi).

22.1 Unambiguous

Matematikte test sorusu çözerken "şununla şunu sadeleştiririm" dersiniz. Örneğin $2T = 4$ için "bu bunu götürür, $T = 2$" dersiniz. Matematikte bu olabilir, ama bilgisayar için bu belirsiz bir şeydir. Çünkü nihai olarak her şeyi komut setinde, CPU'nun anlayacağı dilde yazmamız gerekir. CPU hiçbir zaman "bu bunu götürür" diye bir şey bilmez, ancak siz tarif ederseniz bilir. Tarif de şudur: T'yi yalnız bırakmak için önündeki katsayıyı al, iki tarafı da ona böl.

Burada tarif kesin bir hüküm gibidir. Bilgisayara "yoğurt karadır" diye tarif ederseniz yoğurdu kara yaparsınız. Hoca bir Prolog ödevinden örnek verdi: Bir öğrenci çocuk-ebeveyn ilişkisini yanlış tasarlamıştı ve programda bir çocuğun birden fazla annesi veya babası çıkıyordu. Tarif yanlış olursa bilgisayar hata yapmaz, tarifiniz neyse ona göre çalışır.

> **Hocanın vurgusu:** Algoritma yazarken, özellikle sözde kod (pseudocode) ve akış diyagramında, yalnız temel operasyonlar kullanılır: toplama, çıkarma, çarpma, bölme, eşitlik, karşılaştırma ve atama. "Karesini al", "sırala", "5. kuvvetini al" gibi kısa yollar yanlıştır. Bilgisayar bunları bilmez. Bunların nasıl yapılacağını bu temel işlemler cinsinden tarif etmeniz gerekir.

22.2 Finite

Sonlu olmak, algoritmanın bir yerde tamamlanması demektir. Sonsuza kadar çalışacak bir şey olmamalıdır. Program proses hâline gelip işlemcide işletildiğine göre bu prosesin bir yerde son bulması gerekir. Bu yüzden algoritma sonlu adım içerecek şekilde tasarlanır.

Sonluluğun bir başka yönü de komut setidir. Bir işlemcinin data sheet'ine bakın, içinde belki 700, 800 veya 10.000 komut vardır. 10.000 komut cinsinden yazabilirsiniz. O işlemcide 10.001. bir komut yoksa onu kullanamazsınız. Komut setinin dışında bir şey yazmak hem belirsizlik demektir hem de sonluluğu aşmak demektir.

22.3 Sequence and Set

Hoca bu üç özelliği şu örnekle birleştirdi: Makinemizde A, B ve C adında yalnız üç komut olsun.

- Bütün problemleri A, B ve C ile çözersem **unambiguous** özelliğini sağlarım.
- Bunları belli bir sayıda, örneğin önce A, sonra B, sonra A, sonra B şeklinde, sonsuz olmayacak biçimde sıralarsam **finite** özelliğini sağlarım.
- Bunların belli bir mantıksal sırayla gelmesi **sequence** özelliğidir. Hepsinin bir araya gelmesi de bir küme (set) oluşturur.

Hocaya göre mantıksal sıra belki de en önemli özelliktir. Çünkü programlama dilinin kurallarını, yani noktalı virgülü, döngü ve if tanımını herkes öyle ya da böyle öğrenir. Önemli olan bunların yukarıdan aşağıya mantıklı bir sırayla dizilmesidir.

23. Kek Tarifi Örneği

Sıranın önemini göstermek için hoca derse kek tarifi sordu (önce çay yapımı da konuşuldu: Su önce ısınır, sonra kaynar). Bir öğrencinin verdiği tarif özetle şöyledir:

1. 3 yumurta kırılır ve 1 su bardağı şeker eklenir.
2. Bunlar köpürene kadar çırpılır.
3. Belirli miktarda sıvı yağ ve süt, önceki karışımın içine eklenir ve tekrar çırpılır.
4. Un, kabartma tozu ve diğer kuru malzemeler elenerek eklenir, tekrar çırpılır.
5. İsteğe göre tatlandırıcı bir şeyler eklenir.

Hoca bu tarif üzerinden algoritmanın yapı taşlarını gösterdi:

- **Sıra:** Önce yumurta ve şeker çırpılır, sonra diğerleri eklenir. Köpürmeden diğer malzemeler eklenirse sıra bozulur ve kek kötü olur.
- **Tekrar:** "Köpürene kadar" ifadesi bir süreçten, yani tekrarlı bir işlemden söz eder.
- **Karşılaştırma:** Hoca "un ne kadar eklenecek?" diye sordu. Cevap, bunun yumurtanın büyüklüğüne göre değiştiği ve kendinizin karar vermeniz gerektiğiydi. Yani bir karşılaştırma noktası vardır, bir ölçüsü vardır.

Aynı şey günlük işler için de geçerlidir: Önce çorap, sonra ayakkabı giyilir. Tersi acayip olur. Probleminizin doğru çıktı vermesi için işlemlerin doğru sırada olması gerekir.

Kısacası sıralı (sequence), sonlu ve unambiguous olan, bir problemi çözen işlemlerin tarif edilmesine **algoritma** denir.

24. Üç Temel Yapı

Algoritma deyince karşımıza temelde üç tür işlem çıkar.

> **Hocanın vurgusu:** Algoritmada üç yapı vardır, dört değil. Dördüncüsü yoktur. Bunu benden çok duyacaksınız.

- **Sıralı işlemler (sequential operations)**
- **Şartlı işlemler (conditional operations)**
- **Tekrarlı işlemler (repetitive operations)**

Hoca, uzaktan ders verdiği Azerbaycan'daki bir üniversitede bu yapılar için başka adlar kullanıldığını da aktardı: Sıralıya "hekti", şartlıya "budaklı", tekrarlıya "devirli" yapılar deniyormuş. Oradakilere de hep aynı şeyi söylediğini, üçten başka yapı olmadığını belirtti.

Sıralı işlemler: `a = a + 1` veya `a = 5` gibi işlemlerdir. Program akarken bu satırlara geldiğinde hepsi kesinlikle yapılır.

Şartlı işlemler: Karşımıza `if` gelir. Bir şart yazarız ve belli işlerin yapılıp belli işlerin yapılmayacağını söyleriz:

```
if (şart) {
    A
    B
} else {
    B
    A
}
```

Bir kısım yapılırken diğer kısım yapılmaz. Dikkat edin, bu tekrarlı bir işlem değil, şartlı bir işlemdir.

Tekrarlı işlemler: Belli bir işlemin belli bir duruma göre tekrar tekrar yapılmasıdır. Hoca ikiye ayırdı:

- **Belirli (definite):** Kaç kez tekrar edeceği bellidir. Başlangıcı ve bitişi bellidir: Birden yüze kadar, elliden otuza kadar gibi. Genellikle `for` ile ifade edilir.
- **Belirsiz (indefinite):** Ne zaman biteceği belli değildir. Bulana kadar (until find), bir değerden büyük olana kadar (till greater than) gibi. Genellikle `while` ve `do-while` ile yazılır.

Algoritmada dördüncü bir yapı yoksa programlamada da doğrudan yoktur. Programlama dillerinin sağladığı ekstra kolaylıklara hoca daha sonra değineceğini söyledi.

Dersin Toparlaması

Bu derste makinenin en temelinden gerçek dünya problemlerine doğru bir yol izlendi. Bir bilgisayar temelde CPU, ana bellek ve üç tür bus'tan oluşur. CPU, proses hâline gelmiş programın komutlarını fetch, decode, execute ve write back çevrimiyle tek tek işler. Bu komutlar ve bütün veriler, kayıplı iletimde güvenilir olduğu için elektriğin varlığı ve yokluğu ile, yani 0 ve 1'lerle kodlanır. Her CPU'nun kendine özgü komut seti olduğu için programlama dilleri kullanılır. Karakterler, sayılar, görüntü, ses ve video bu 0-1'lerle belli kurallara göre saklanır. Boyutları da bit sayısı üzerinden hesaplanır. Analog sinyaller ise zamanda örnekleme, genlikte nicemleme ile sonlu hâle getirilir.

Dersin sonunda algoritma, girişten çıkışı üreten fonksiyonu bulma süreci olarak tanımlandı. Özellikleri unambiguous, finite, sequence and set olarak verildi. Yapı taşlarının sıralı, şartlı ve tekrarlı olmak üzere üç tane olduğu vurgulandı. Sonraki derste algoritmanın nasıl geliştirileceği örneklerle görülecek, iyi algoritmanın ne olduğu konuşulacak ve bunun C dilinde nasıl gerçekleştirileceği üzerinde çalışılacak.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.